

K L H EDUCATION FOUNDATION
DATA STRUCTURES & ALGORITHMS – 3
PROJECT REPORT
DICTIONARY-BASED MISPELLED WORD
DETECTION SYSTEM

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

A.Y. 2026-2027

Submitted by

Hansika Reddy Gunapati – 2520030237

Teja Hasini Reddy – 2520030048

Anthony Nissy – 2520030064

Team Number: 16 | Section: 7

Under the guidance of

Dr. S. Madhavi

Department of Computer Science Engineering



Hyderabad-500090, Telangana, India.



DECLARATION

We hereby declare that the project report entitled “Dictionary-Based Misspelled Word Detection System” is a record of bona fide work carried out by Hansika Reddy Gunapati (2520030237), Teja Hasini Reddy (2520030048), and Anthony Nissy (2520030064), submitted in partial fulfilment of the requirements for the course Data Structures and Algorithms -3 (25CS2103E) during the academic year 2026-2027.

The work presented in this report is based on the implementation developed by the project team. The report describes the data structures, string-matching algorithms, correction techniques, evaluation procedure, and web demonstration implemented for the project.

We further declare that the work embodied in this report has not been knowingly copied from another department, university, or institute and that the sources used for background study have been acknowledged in the References section.

Hansika Reddy Gunapati – 2520030237

Teja Hasini Reddy – 2520030048

Anthony Nissy – 2520030064

K L H (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project-based report entitled “Dictionary-Based Misspelled Word Detection System” is a bona fide work done and submitted by Hansika Reddy Gunapati (2520030237), Teja Hasini Reddy (2520030048), and Anthony Nissy (2520030064) in partial fulfilment of the requirements for the course Data Structures and Algorithms -3 (25CS2103E), during the academic year 2026-2027.

The project demonstrates the application of data structures and string algorithms to dictionary-based spelling error detection, correction suggestion, pattern matching, evaluation, and interactive visualization.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the support and guidance received from many people. We sincerely express our gratitude to everyone who contributed directly or indirectly to the completion of this Data Structures and Algorithms -3 project.

We express our deepest appreciation to our guide, Dr. S. Madhavi, Department of Computer Science Engineering, for providing valuable guidance, technical suggestions, encouragement, and continuous support throughout the project development.

We thank the faculty members and the Department of Computer Science Engineering of KLV Education Foundation for providing the academic environment and facilities required to carry out the project. We are also grateful for the learning resources and laboratory infrastructure that supported the implementation and testing of the system.

We acknowledge the importance of the Data Structures and Algorithms course in helping us apply concepts such as Tries, string matching, dynamic programming, pattern searching, and algorithm evaluation to a practical problem.

Finally, we thank our parents, friends, classmates, and all those who assisted us with feedback, testing, and encouragement during the development of the Dictionary-Based Misspelled Word Detection System.

Hansika Reddy Gunapati

Teja Hasini Reddy

Anthony Nissy

TABLE OF CONTENTS

S.No.	Contents	Page No.
1.	Abstract	6
2.	Introduction	7
3.	Literature Survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software Requirements	10
6.	Implementation	11
7.	Results & Discussion	13
8.	Conclusion & Future Scope	15
9.	References	16

Note: Frontend application screenshots are intentionally left as placeholders in the Results & Discussion section and can be inserted after the report is reviewed.

1. ABSTRACT

The Dictionary-Based Misspelled Word Detection System is a practical Data Structures and Algorithms application designed to detect misspelled words and generate ranked correction suggestions for user-entered text. The system uses a dictionary containing 370,105 words and 979 common words, with a Trie providing efficient dictionary membership testing. When an input token is not found, dynamic programming based edit-distance computation is used to measure similarity with candidate words, while Soundex and common-word information support suggestion ranking and tie-breaking. The project also implements KMP string matching and Aho–Corasick multi-pattern matching to demonstrate classical string-processing techniques, together with Dijkstra shortest-path and Edmonds–Karp maximum-flow modules for graph-based algorithm demonstrations.

The system is implemented in Java for the core DSA functionality and complemented by a browser-based frontend for interactive visualization. The processing pipeline consists of text tokenization, Trie lookup, error identification, candidate generation, distance calculation, ranking, and result presentation. The console application reports the detected status, top suggestions, edit distances, and execution time, while the frontend displays highlighted errors, ranked suggestions, and algorithm-processing information. Evaluation on the supplied labelled test data produced 100% detection precision, 90% recall, and an F1-score of 94.7%, with top-1 and top-3 suggestion accuracies of 86.7% and 90.0%, respectively. The project demonstrates how fundamental data structures, string algorithms, dynamic programming, and graph algorithms can be integrated into a single application to address a real-world text-processing problem while providing measurable performance and an interactive learning-oriented interface.

TEAM DETAILS

Team 16 | Section 7

Hansika Reddy Gunapati – 2520030237

Teja Hasini Reddy – 2520030048

Anthony Nissy – 2520030064

Guide: Dr. S. Madhavi

2. INTRODUCTION

Spelling errors are common in user-generated text, documents, logs, and other digital content. Although a single misspelled word may appear harmless, repeated errors can reduce readability, affect search and text analytics, and introduce noise into downstream natural-language processing tasks. The project therefore focuses on a practical problem: detecting words that are absent from a predefined dictionary and producing useful correction suggestions.

The central idea is to combine an efficient dictionary data structure with string-processing algorithms. A Trie stores dictionary words character by character and supports direct membership testing by following a path through the input characters. When a word is not found, edit-distance dynamic programming provides a measurable similarity score for candidate corrections. This division between exact lookup and approximate comparison avoids repeatedly scanning the complete dictionary for every input word.

The project has both a Java console implementation and a browser-based frontend. The Java implementation provides dictionary loading, Trie lookup, spell checking, suggestion generation, KMP, Aho–Corasick, algorithm comparison, graph demonstrations, and labelled-data evaluation. The frontend mirrors the algorithmic ideas with JavaScript modules and interactive visualizations. The two parts therefore provide both an executable DSA implementation and a visual learning environment.

The overall processing flow is: input text → tokenization → dictionary/Trie lookup → error identification → candidate generation → edit-distance and heuristic ranking → ranked suggestions → evaluation/display. For multi-pattern tasks, the Aho–Corasick automaton provides a separate path from the ordinary spell-checking flow. Graph modules use the same project data as an educational model for shortest-path and maximum-flow computation.

This project is particularly relevant to Data Structures and Algorithms because the practical system depends on the behavior and complexity of Tries, string matching, dynamic programming, and graph algorithms. It also demonstrates an important engineering principle: the best algorithm is not selected in isolation, but according to the operation that must be performed, the size of the input, and the information that must be returned.

3. LITERATURE SURVEY

1. Cormen, Leiserson, Rivest and Stein — Introduction to Algorithms

Cormen, Leiserson, Rivest and Stein provide the general foundations for algorithm design, asymptotic analysis, dynamic programming, graph algorithms, and data structures. This reference supports the complexity-oriented discussion in this report and the graph-processing demonstrations included in the project.

Reference: T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

2. Sedgewick and Wayne — Algorithms

Sedgewick and Wayne present practical algorithms for searching, graphs, symbol tables, and algorithm analysis. Their treatment is useful for relating the project implementation to standard data-structure operations and for explaining the importance of choosing an appropriate representation for a given task.

Reference: R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley Professional, 2011.

3. Gusfield — Algorithms on Strings, Trees, and Sequences

Gusfield develops exact string matching, suffix-based structures, inexact matching, sequence alignment, and dynamic programming. The book is directly relevant to spelling correction because a misspelled word is naturally treated as a string-comparison problem.

Reference: D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

4. Crochemore, Hancart and Lecroq — Algorithms on Strings

Crochemore, Hancart and Lecroq discuss pattern-matching automata, string searching, indexes, alignments, and approximate patterns. This provides a theoretical reference for KMP-style searching, automata-based matching, and edit-distance related processing used in the system.

Reference: M. Crochemore, C. Hancart and T. Lecroq, Algorithms on Strings, Cambridge University Press, 2007.

5. Aho and Corasick — Multiple-pattern string matching

Aho and Corasick introduced the classical automaton-based method for locating multiple patterns efficiently in a text. The project applies this idea through a Trie with failure/output links so several patterns can be processed during one scan of the text.

Reference: A. V. Aho and M. J. Corasick, “Efficient string matching: an aid to bibliographic search,” Communications of the ACM, 18(6), 333–340, 1975.

6. Practical spell-checking systems — Hunspell, LanguageTool, SymSpell, Norvig and GNU Aspell

Practical spell-checkers demonstrate complementary strategies such as dictionary and affix processing, linguistic rules, fast candidate generation, edit-distance correction, and frequency-aware ranking. These systems were studied as application references, while the present project keeps its Java core focused on DSA concepts that can be implemented, explained, and measured directly.

Comparative synthesis: the academic references show a progression from fundamental algorithm analysis to specialized string-processing techniques and then to complete spell-checking systems. The present project adopts the transparent DSA components from these foundations—Trie lookup, KMP, Aho–Corasick, dynamic programming and graph algorithms—while avoiding dependence on a third-party spell-checking engine. This makes the implementation suitable for demonstrating both algorithmic reasoning and practical system integration.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Develop an efficient dictionary-based application that identifies misspelled words in text, suggests plausible corrections, and demonstrates classical string and graph algorithms through a menu-driven Java console and interactive browser frontend.

Problem Description: Large text inputs can contain many spelling errors, and repeatedly comparing each word against every dictionary entry is computationally expensive. The proposed solution should provide fast dictionary lookup, efficient pattern matching, useful correction suggestions, and measurable evaluation. It should also expose the underlying DSA operations so that the project remains educational rather than behaving as a black-box spelling service.

OBJECTIVES

- Implement dictionary lookup for misspelled-word detection using a Trie and analyze its lookup behavior.
- Detect errors in typed text and text files and produce ranked correction suggestions.
- Implement KMP for exact single-pattern matching and expose its LPS table/search behavior.
- Implement Aho–Corasick for multi-pattern matching using Trie and failure-link processing.
- Rank correction candidates using edit-distance-based dynamic programming and supporting heuristics.
- Provide an evaluation module using labelled test data and report precision, recall, F1, top-1, and top-3 measures.
- Provide a browser-based dashboard for interactive algorithm demonstration, visualization, and comparison.
- Demonstrate network-flow based suggestion matching using Edmonds–Karp and related flow processing in the frontend.
- Measure execution behavior and discuss limitations such as dictionary ambiguity, uncommon short words, and lack of sentence-level context.

Functional scope: the user can load the dictionary, enter a word or sentence, obtain a spelling status, view ranked suggestions, and access separate algorithm demonstrations. Evaluation is performed on labelled test data so that the system can report measurable detection and suggestion performance rather than only showing a single successful example.

Design constraints: the solution is intended to remain within the DSA project environment, use explicit data structures and algorithms, operate with a large local dictionary, and expose understandable intermediate results. The project therefore emphasizes deterministic algorithmic processing and readable console output instead of relying on an external cloud API or a hidden correction service.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Computer/Laptop with a modern multi-core processor.
- Minimum 4 GB RAM; 8 GB or more is recommended for comfortable IDE/browser use.
- At least 1 GB free storage for source files, dictionary corpus, and generated build files.
- Keyboard, display, and browser for interactive frontend testing.

Software Requirements

- Java Development Kit (JDK 11 or later; project documentation specifies JDK 11+).
- IntelliJ IDEA / Visual Studio Code or another Java-compatible IDE.
- Modern web browser such as Chrome, Edge, Firefox, or Safari.
- HTML5 and CSS3 for the frontend interface.
- JavaScript for Trie, KMP, Aho–Corasick, suggestion, comparison, and network-flow visualizations.
- Project corpus containing dictionary.txt, common_words.txt, input documents, and labelled test_data.csv.
- No external Java libraries are required by the supplied Java implementation.

Development Environment

Development Environment: The Java project is organized into source modules including DictionaryLoader, Trie, SpellChecker, SuggestionEngine, KMP, AhoCorasick, Evaluator and Main. The corpus directory contains the dictionary and supporting test data. The browser project separates algorithm demonstrations into JavaScript modules for Trie, KMP, Aho–Corasick, suggestion ranking, comparison, and graph-flow visualization. The supplied Java implementation requires no external Java libraries, keeping the core experiment reproducible in a standard JDK environment.

Input and output expectations: the system accepts individual words as well as sentence-like text for spell checking. For an incorrect token, the output identifies the token, marks it as incorrect, and presents ranked alternatives with distance information. The evaluation module additionally reads labelled examples and reports aggregate measures.

Compatibility and reproducibility: the project can be executed in a standard Java IDE with the supplied corpus files placed at the expected relative paths. The frontend can be opened in a modern browser without requiring a JavaScript framework. Keeping the algorithm modules self-contained makes the demonstrations easier to inspect, test, and present during project evaluation.

6. IMPLEMENTATION

The implementation is organized into modules so that each major DSA technique has a clear responsibility. The Java console program forms the core processing layer, while the frontend provides an interactive representation of the same ideas. The design follows a pipeline: load corpus → construct data structures → accept input → perform exact lookup → identify errors → generate and rank candidates → display results → evaluate performance. This modular structure makes each algorithm independently testable and makes its role in the complete system easier to explain.

Module 1 – Dictionary Loading and Trie Lookup

DictionaryLoader reads the dictionary corpus and initializes the spelling system. The Trie stores each word character-by-character, with a terminal marker identifying complete words. For a query of length L , traversal is proportional to L once the structure is built. The supplied console execution reports 370105 dictionary words and 979 common words, demonstrating that the implementation is designed to work with a substantial lexicon rather than a small hand-written list.

Module 2 – Spell Checking and Suggestion Engine

SpellChecker tokenizes input, validates words through the dictionary/Trie, records error locations, and requests suggestions from the suggestion engine. Candidate ranking uses edit-distance based dynamic programming together with supporting heuristics such as common-word preference and phonetic similarity. The distance value gives a transparent measure of character-level difference, while ranking heuristics help decide between candidates with similar distances.

Module 3 – String Matching: KMP and Aho–Corasick

KMP builds an LPS (Longest Prefix Suffix) table for one pattern and searches a text in $O(n+m)$ time. The preprocessing table allows the search to reuse information after a mismatch instead of restarting from the next text character. Aho–Corasick builds a Trie of multiple patterns and adds failure/output links so the text can be scanned once while reporting matched patterns. Together they demonstrate optimized single-pattern and multi-pattern processing.

Module 4 – Evaluation and Interactive Web Algorithms

The Evaluator processes labelled test pairs and calculates detection precision, recall, F1-score, top-1 suggestion accuracy, and top-3 suggestion accuracy. The frontend additionally visualizes algorithm comparisons and models correction assignment as a bipartite maximum-flow problem using Edmonds–Karp and Dinic’s algorithm. These modules extend the project beyond simple lookup and show how different DSA techniques can coexist in one application.

Integrated data flow: dictionary loading happens once at startup, after which the Trie and Aho–Corasick structures are available for repeated queries. A normal spell-check request first performs direct lookup; only an absent token requires correction processing. This staged design prevents the expensive edit-distance computation from being treated as the first operation for every word. The result is then formatted for the console or represented through the frontend visualizations.

DSA learning coverage: the implementation demonstrates dictionary indexing through a Trie, exact pattern matching through KMP, multi-pattern matching through Aho–Corasick, dynamic programming through edit distance, and graph processing through Dijkstra and Edmonds–Karp. These components provide a direct link between classroom algorithm concepts and a working application, while the evaluation module supplies measurable evidence of system behavior.

Complexity Summary

Technique	Main Operation	Complexity
Trie	Lookup / insert	$O(L)$
Edit Distance	DP candidate comparison	$O(p \times q)$
KMP	Single-pattern search	$O(n+m)$
Aho–Corasick	Multi-pattern search	$O(n+z)$
Dijkstra	Shortest path demonstration	$O((V+E) \log V)$
Edmonds–Karp	Maximum-flow demonstration	$O(VE^2)$

7. RESULTS & DISCUSSION

The revised console execution screenshots supplied with the project provide direct evidence of the running Java application. The first screenshot shows successful dictionary loading, construction of the Trie and Aho–Corasick structures, and the complete menu of implemented DSA demonstrations. It reports 370105 dictionary words and 979 common words. The second screenshot records a normal spell-check run using the input “hhhl0”. The system marks the input as INCORRECT and produces three ranked suggestions.

```
C:\Program Files\Java\jdk-20\bin\java.exe" --javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.3\lib\idea_rt.jar=3542
Dictionary loaded successfully.
Dictionary path: C:\Users\adoor\OneDrive\Documents\CODING\DSA3\PROJECT_2-1\corpus\dictionary.txt
Dictionary words: 370105
Common words loaded: 979

Building Trie and Aho-Corasick structures...
Trie and Aho-Corasick structures ready.

=====
          DSA SPELL CHECKER SYSTEM
=====
Dictionary words loaded: 370105

1. Normal Spell Check
2. KMP Spell Check
3. Levenshtein Edit Distance Check
4. Aho-Corasick Pattern Matching
5. Trie Dictionary Search
6. Algorithm Comparison
7. Dijkstra Shortest Path
8. Edmonds-Karp Maximum Flow
9. Test Data Evaluation
10. Exit
Enter your choice: |
```

Figure 7.1 Console execution: dictionary initialization and DSA Spell Checker menu

```
Enter your choice: 1

Enter word or sentence to check: hhhlo

=====
                SPELL CHECK RESULT
=====

Input: hhhlo
Status: INCORRECT
Top Suggestions:
    1. hello (Distance: 2)
    2. who (Distance: 3)
    3. help (Distance: 3)

Execution Time: 155.6460 ms
```

Figure 7.2 Console spell-check result for input “hhhl0”

The observed sample run reports “hello” as the top suggestion with edit distance 2, while “who” and “help” are listed at distance 3. The displayed execution time is 155.6460 ms for that console operation. Separately, the project documentation reports results on a 70-pair labelled test set: detection precision 100%, recall 90%, F1-score 94.7%, top-1 suggestion accuracy 86.7%, and top-3 suggestion accuracy 90.0%. These values are project-test results and should be interpreted as evidence from the included labelled set rather than as a general benchmark.

Frontend Outputs

Quick Test Presets:

Preset 1: Demo ("wat is yoe nama...")Preset 2: Project TyposPreset 3: Frequent ErrorsPreset 4: Flow Collisions

HeLo hapy borthday

Auto-Apply All Top-1 Corrections

Dictionary-Based Spell Checker (Trie + Edit Distance)

Tokenizes input text and queries each token in the Trie (O(L)). When a word is missing from the lexicon, the SuggestionEngine calculates Damerau-Levenshtein Edit Distance via Dynamic Programming with Soundex phonetic tie-breaking, ranking the Top 3 Suggestions for instant replacement.

TOTAL WORDS

3

ERRORS DETECTED

3

TEXT ACCURACY

0.0%

TRIE + DP TIME

284.00 ms

Document Highlighted Preview

HeLo hapy borthday

Spell Checker Dashboard

Top 3 Ranked Suggestions per Detected Error

Click any suggestion pill to replace in text

Error #1 "HeLo" Index: 0

#1 hello dist: 1
Soundex Match Common Word
Click to Replace ↕

#2 help dist: 1
Common Word
Click to Replace ↕

#3 haLo dist: 1
Soundex Match
Click to Replace ↕

Error #2 "hapy" Index: 5

#1 happy dist: 1
Soundex Match Common Word
Click to Replace ↕

#2 hap dist: 1
Soundex Match
Click to Replace ↕

#3 hapi dist: 1
Soundex Match
Click to Replace ↕

Error #3 "borthday" Index: 10

#1 birthday dist: 1
Soundex Match
Click to Replace ↕

#2 birthdays dist: 2
Soundex Match
Click to Replace ↕

#3 bothway dist: 2
Click to Replace ↕

Top-3 Suggestions

8. CONCLUSION & FUTURE SCOPE

The Dictionary-Based Misspelled Word Detection System successfully demonstrates how classical data structures and algorithms can be combined to solve a practical text-processing problem. The Trie provides efficient dictionary membership testing, dynamic programming supplies an explicit edit-distance measure for approximate comparison, KMP demonstrates efficient single-pattern matching, and Aho–Corasick demonstrates multi-pattern matching with failure links. The graph modules further demonstrate shortest-path and maximum-flow techniques within the same academic project.

The project also provides an interactive frontend so that algorithmic behavior can be understood visually rather than only through source code. The console results and reported evaluation metrics show that the system can detect errors and produce useful ranked suggestions, while also making the computational trade-offs visible. The main limitation is that character-level similarity does not provide full sentence context; future work can therefore add language-model or n-gram based ranking, larger multilingual dictionaries, and more extensive evaluation datasets.

Future Scope

- Add context-aware correction using n-gram or language-model based ranking.
- Support larger and frequency-weighted multilingual dictionaries.
- Improve handling of apostrophes, named entities, abbreviations, and domain-specific vocabulary.
- Introduce persistent user dictionaries and configurable correction rules.
- Extend benchmarking to larger labelled corpora and compare more candidate-generation methods.
- Integrate the Java engine with a production-style web API instead of keeping the frontend demonstration self-contained.
- Improve the network-flow assignment objective by incorporating suggestion confidence/costs rather than only unit-capacity matching.

9. REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022. ISBN 978-0-262-04630-5.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011. ISBN 978-0-321-57351-3.
- [3] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge, U.K.: Cambridge University Press, 1997. ISBN 978-0-521-58519-4. doi: 10.1017/CBO9780511574931.
- [4] M. Crochemore, C. Hancart, and T. Lecroq, *Algorithms on Strings*. Cambridge, U.K.: Cambridge University Press, 2007. ISBN 978-0-521-84899-2. doi: 10.1017/CBO9780511546853.
- [5] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, Jun. 1977. doi: 10.1137/0206024.
- [6] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, Jun. 1975. doi: 10.1145/360825.360855.
- [7] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966. English translation of the 1965 Russian original.
- [8] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, Mar. 1964. doi: 10.1145/363958.363994.
- [9] R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, Jan. 1974. doi: 10.1145/321796.321811.
- [10] P. Norvig, “How to Write a Spelling Corrector,” 2007. Available: <https://www.norvig.com/spell-correct.html>. Accessed: Oct. 6, 2026.
- [11] Hunspell, “Hunspell: A spell checker and morphological analyzer,” open-source project. Available: <https://github.com/hunspell/hunspell>. Accessed: Oct. 6, 2026.
- [12] Oracle, “Java Platform, Standard Edition Documentation,” Java SE documentation. Available: <https://docs.oracle.com/en/java/javase/>. Accessed: Oct. 6, 2026.
- [13] Project Team, “Dictionary-Based Misspelled Word Detection System,” Java source code, corpus, test data, and frontend implementation, supplied project repository, 2026.